

Hermes Personal OS — Complete Implementation Guide

Version: 0.1.0
Generated: June 2026
Repository: hermes-personal-os (local-first proactive assistant)

Table of Contents

- 1. [Executive Summary](#)
 - 2. [Architecture Overview](#)
 - 3. [Bootstrap & Runtime](#)
 - 4. [Chat Server & UI](#)
 - 5. [Live Logs](#)
 - 6. [Orchestrator & Message Flow](#)
 - 7. [Agents](#)
 - 8. [Autonomous Agent Loop](#)
 - 9. [Steering & Parallel Tasks](#)
 - 10. [Tool Executor & Approvals](#)
 - 11. [Prompts Reference](#)
 - 12. [Configuration Files](#)
 - 13. [Proactive & Background](#)
 - 14. [iMessage Bridge](#)
 - 15. [Environment Variables](#)
 - 16. [Data Files](#)
 - 17. [File Index](#)
-

1. Executive Summary

Hermes Personal OS is a **local-first proactive assistant** that runs on your Mac. It does not rely on Gmail or social APIs for core workflows; instead it uses **Arc browser + screen capture + Cloudflare Workers AI vision** to read and act on digital surfaces.

Primary user interfaces today:

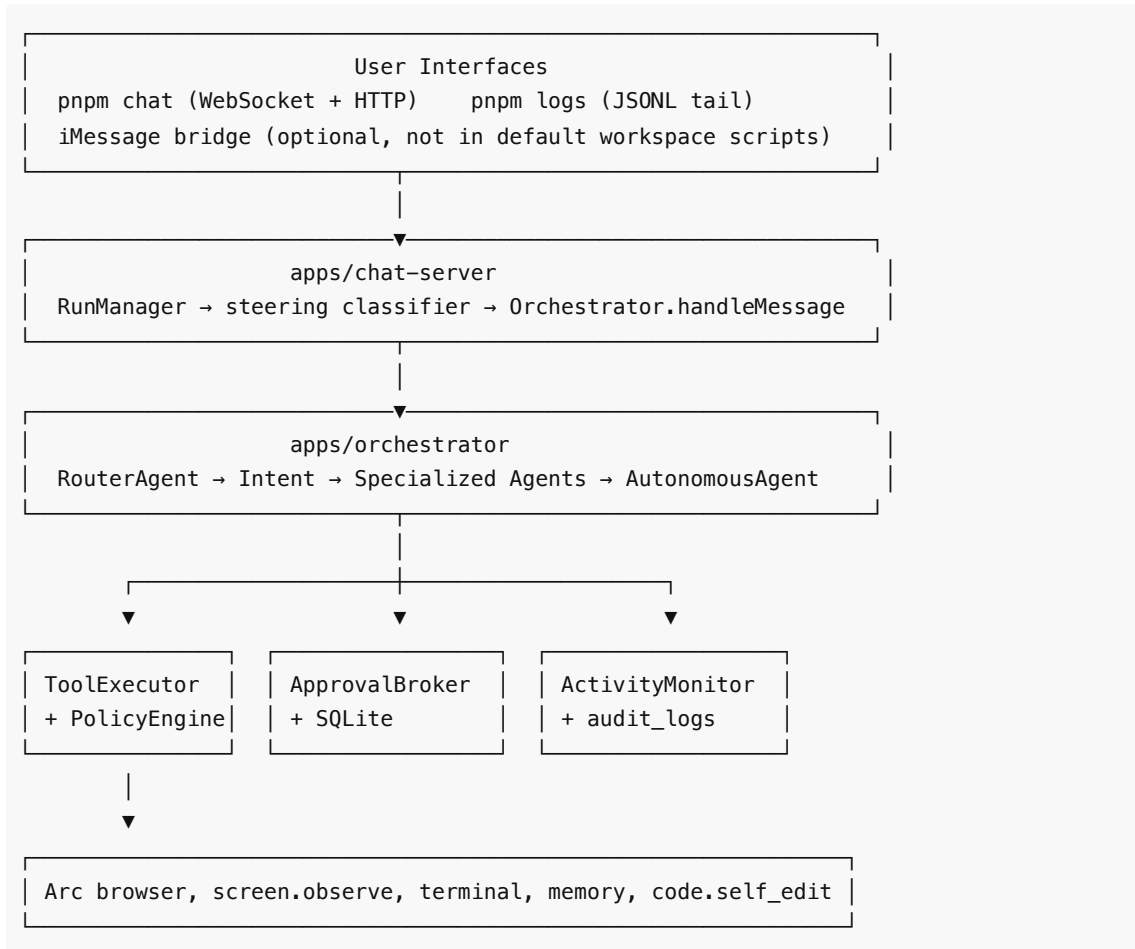
Interface	Command	Purpose
Chat UI	pnpm chat	Web chat at http://127.0.0.1:3847 with steering and parallel tasks
Live logs	pnpm logs	Tail data/activity.jsonl with formatted agent/tool lines

Core capabilities:

- Intent routing via LLM classifier
- Multi-step autonomous agent (think → tool → observe → replan)
- Approval-gated high-risk actions (tweets, iMessage, code edits)
- Mid-task **steering** (related messages pause planning and replan from current trace)
- **Parallel tasks** (unrelated messages run separately while primary task continues)
- Research sessions with link follow-ups
- Proactive scans (Gmail, X, LinkedIn, Calendar) when iMessage bridge runs

- Hybrid memory (SQLite + optional Supermemory)

2. Architecture Overview



Workspace packages (pnpm)

Package	Role
@hermes-os/shared	Types, Cloudflare client, intent, assistant policy, SteerController
@hermes-os/orchestrator	Orchestrator, all agents, proactive scheduler
@hermes-os/chat-server	HTTP/WebSocket chat UI
@hermes-os/tool-executor	Tools + macros
@hermes-os/context-graph	SQLite repos
@hermes-os/audit-log	Audit + activity formatting
@hermes-os/approval-broker	Approval workflow
@hermes-os/policies	Risk + proactivity YAML loaders
@hermes-os/memory	Hybrid memory

@hermes-os/connectors	Connector hub
@hermes-os/risk-engine	Risk detectors

3. Bootstrap & Runtime

File: apps/orchestrator/src/system.ts — bootstrapPersonal0s()

Startup sequence:

1. Resolve workspace root (HERMES_OS_ROOT or auto-detect)
2. Load .env
3. Open SQLite (data/local.sqlite), run migrations
4. Create repositories (approvals, audit, state, memories, tasks, open loops, sources, risks)
5. AuditLogger + ActivityMonitor (mirrors to JSONL + optional console)
6. HindranceCoordinator (system-level pause notifications)
7. ApprovalBroker (TTL from APPROVAL_TTL_SECONDS , default 300s)
8. PolicyEngine loads configs/risk-policy.yaml
9. Hybrid memory + Hermes gateway + Cloudflare Workers AI clients
10. createToolRegistry() + ToolExecutor
11. IntentClassifier (if Cloudflare configured)
12. createAgentRuntime() → Orchestrator

Chat entry: apps/chat-server/src/index.ts

```
const system = bootstrapPersonal0s();
startChatServer(system);
```

4. Chat Server & UI

Server (apps/chat-server/src/server.ts)

Setting	Default
Host	127.0.0.1
Port	3847 (HERMES_CHAT_PORT)
Auto-open browser	macOS open URL (disable: HERMES_CHAT_NO_OPEN=1)

HTTP endpoints:

- GET / — Chat SPA (public/index.html)
- GET /api/status — Run manager status JSON
- POST /api/chat — { "text": "... " } submit message

WebSocket events (client → server):

Message	Action
{ type: "message", text }	Submit user message

{ type: "interrupt" }	Hard-stop primary task
{ type: "status" }	Refresh status

WebSocket events (server → client):

Event	Meaning
status	running, runId, goal, parallelTasks
run_started	New run began
run_finished	Run completed
steering_applied	Related message injected into primary task
parallel_task	Unrelated message started as separate run
interrupted	Primary task aborted
reply	Final assistant text for a run
error	Run error
activity	Live formatted activity line

Run Manager (apps/chat-server/src/run-manager.ts)

Decision tree when user sends a message:

```

Is there a primary run?
NO  → Start new primary run (with SteerController)
YES → Is message related to primary goal?
      YES → steerController.requestSteer(message)
            Emit steering_applied
            Primary continues (no abort, no tab close)
      NO  → Start parallel run (no SteerController)
            Primary keeps running
            Emit parallel_task

```

Interrupt (Stop button): Aborts primary `AbortController`, clears `SteerController`, does not affect parallel runs.

Each run calls:

```

orchestrator.handleMessage(
  { channel: "web", senderId: "web-user", text, receivedAt },
  { signal: controller.signal, steerController }
);

```

Chat UI (apps/chat-server/public/)

- **Layout:** Chat panel (left) + live activity stream (right)
- **Composer:** Enter to send, Shift+Enter for newline

- **Stop:** Sends `interrupt` WebSocket message
- **Steering UX:** System message explains related vs parallel behavior

5. Live Logs

Script: `scripts/tail-logs.mjs`

Command: `pnpm logs` or `pnpm logs 100` (last N lines)

Source file: `data/activity.jsonl` (one JSON object per line)

Formatted line types:

Prefix	Event
USER	Incoming message
REPLY	Outgoing assistant message
AGENT ►	Agent invoked
STEP	Agent step (think/tool)
AGENT ✓/✗	Agent finished
PAUSE	Agent blocked (needs user)
TOOL ?/✓/✗	Tool requested/executed/denied
ROUTE	Intent classified
SCAN	Digital presence scan

Activity monitor (`packages/audit-log/src/activity-monitor.ts`):

- Mirrors all `AuditLogger` events
- Writes JSONL (default on; `HERMES_ACTIVITY_JSONL=0` disables)
- Console when `HERMES_ACTIVITY_CONSOLE=1`
- WebSocket subscribers (chat UI sidebar)

6. Orchestrator & Message Flow

File: `apps/orchestrator/src/orchestrator.ts`

`handleMessage(message, options?)` pipeline

1. `throwIfAborted(signal)` – hard cancel only
2. Audit: `incoming_message`
3. Check assistant state:
 - `paused` → "Assistant is paused"
 - `emergency_stop` → "Restart orchestrator"
4. Early resume handlers (before routing):
 - a. `AutonomousAgent.tryHandleUserGuidance` → `pending-blocked.json`
 - b. `HindranceCoordinator.tryResumeFromUser` → `pending-hindrance.json`

```

c. LaptopControlAgent.tryHandleCredentialReply → pending-login.json
5. maybeRememberPreferences (auto-capture sleep/pillow prefs)
6. tryDirectCommand:
  - daily brief / morning brief
  - evening review
  - logs / activity / monitor [n]
  - research <topic>
7. RouterAgent.classify(text, context)
8. Session bias: research follow-up, shopping→research
9. Audit: intent_classified
10. tryAssistantControl: pause, resume, emergency_stop, status
11. Switch intent → agent handler
12. recoverIfRefusal (links, research, autonomous fallback)
13. Audit: outgoing_message

```

Intent routing table

Intent	Handler
approval_response	ApprovalAgent
research	ResearchAgent + session state
memory_update	MemoryAgent
personal_ops	AutonomousAgent OR ChiefOfStaff morning brief
browser_task	AutonomousAgent
coding	CodingAgent
writing	WritingAgent
laptop_control	Operator / Autonomous / LaptopControl
unknown	Research / Autonomous / GeneralAgent
status	buildStatus()

Autonomous routing triggers

From `packages/shared/src/assistant-policy.ts` :

- `needsBrowserAutonomy(text)` — gmail, inbox, twitter, linkedin, calendar, etc.
- `needsAgentPlanner(text)` — browser autonomy OR shopping/links OR action verbs
- `shouldRunAutonomous()` — AutonomousAgent exists AND planner needed

7. Agents

Agent	File	Requires CF?	Role
RouterAgent	router-agent.ts	Via classifier	Intent classification

ResearchAgent	research-agent.ts	Yes	Deep research + sessions
MemoryAgent	memory-agent.ts	Optional	remember/forget/search
ChiefOfStaffAgent	chief-of-staff-agent.ts	Optional	Morning/evening briefs
AutonomousAgent	autonomous-agent.ts	Yes	Multi-step Mac operator
DigitalPresenceMonitor	digital-presence-monitor.ts	Vision	Arc scans Gmail/X/LinkedIn/Calendar
CodingAgent	coding-agent.ts	CF or Hermes	Code + self_edit
BrowserAgent	browser-agent.ts	Optional	Headless fetch summary
LaptopControlAgent	laptop-control-agent.ts	Optional	Screen + browser steps
WritingAgent	writing-agent.ts	Optional	Draft messages
GeneralAgent	general-agent.ts	Yes	Fallback conversation
ApprovalAgent	approval-agent.ts	No	Approve/deny/edit
OperatorAgent	operator-agent.ts	No	Direct tool plans

8. Autonomous Agent Loop

File: apps/orchestrator/src/autonomous-agent.ts

Config: configs/agent-loop.yaml

```
max_steps: 12
replan_on_failure: true
auto_observe_after_browser: true
allow_self_edit_when_stuck: true
```

Per-step algorithm

```
FOR step = 0 .. max_steps-1:
  1. throwIfAborted(signal) # hard stop only
  2. steerController.takeSteering() → inject into steeringNotes + trace
  3. Build planner prompt (goal, trace, errors, tools, screen path, hints)
  4. CF chat with planner system prompt + assistant policy
  5. Post-think steer check (if steer arrived during think, replan same step)
  6. Parse JSON planner step
  7. If ask_user/blocked → save pending-blocked.json, return "Paused"
  8. If finish → return final answer + trace
  9. Invoke ONE tool via ToolExecutor
  10. If browser.goto + auto_observe:
      - Wait 2.5s
```

```
- screen.observe
- analyzeScreenForContext (vision + structure)
11. On repeated errors → code.self_edit or blockAndAskUser
```

Blocked session resume

Store: data/pending-blocked.json

When user replies, `tryHandleUserGuidance()` runs before normal routing:

```
hint = "User guidance: {text}\nPrior trace:\n{trace}"
resume run(goal, { hint, resumeFromBlocked: true })
```

9. Steering & Parallel Tasks

SteerController (packages/shared/src/steer-controller.ts)

Cooperative pause **between steps** — does NOT:

- Close browser tabs
- Abort in-flight tool calls
- Restart goal from scratch

Does:

- Queue user message for next planner iteration
- Inject as User course-correction (keep browser/tabs as-is, replan next step): ...

Steering relevance classifier

File: apps/orchestrator/src/steering-classifier.ts

Heuristics (fast path):

- Phrases: instead, wait, actually, stop, don't, focus on, try, etc.
- Token overlap ≥ 2 between goal and new message
- Short messages with ≥ 1 shared keyword

LLM fallback:

```
Active task: {goal}
New user message: {msg}
Does the new message steer, correct, or add to the active task?
Reply ONLY JSON: {"related":true|false,"reason":"..."}
```

Orchestrator API: `orchestrator.isSteeringRelated(activeGoal, newMessage)`

10. Tool Executor & Approvals

Registered tools

Tool	Description
<code>memory.remember / forget / search</code>	Hybrid memory

filesystem.read / write	Workspace-scoped files
screen.observe	macOS screenshot
browser.goto	Open URL in Arc
browser.fill_credentials	Login assist
web.fetch	HTTP fetch
terminal.run	Shell in workspace
social.post	Post via Arc (approval)
code.self_edit	Hermes gateway repo edits
imessage.send	Messages.app
tools.define / tools.author / tools.run	Custom macros

Execution flow

```
PolicyEngine.evaluate(tool, context)
  → deny if unknown (default deny_if_unknown)
  → pending_approval if requiresApproval && no approvalId
  → verify payload hash if approvalId present
  → execute tool
  → audit tool_call_executed
```

Approval summary (configs/risk-policy.yaml)

Level	Examples
always	social.post, imessage.send, code.self_edit, filesystem.delete
true	filesystem.write, tools.author, calendar.create
if_destructive	terminal.run, tools.run
if_semantic_risk	browser.click
false	screen.observe, browser.goto, memory.*, web.fetch

11. Prompts Reference

11.1 Global Assistant Policy

File: packages/shared/src/assistant-policy.ts

```
CRITICAL – Hermes Personal OS behavior:
– NEVER say you cannot browse, cannot access the internet, or are unable to help if any tool can apply.
– You HAVE: research, Arc browser (browser.goto for Gmail/X/LinkedIn – no API keys), screen capture, web.fetch, code.self_edit, custom tools, memory.
```

- If stuck: replan, try `code.self_edit`, then ask the user one clear question and wait.
- If the user wants a link: provide `https://` URLs (Amazon search links are fine). Repeat links from the conversation if already given.
- If something needs live browsing: say you are opening Arc or use research links – do not refuse.
- Prefer doing something useful over apologizing. Only say no when a tool returned `denied/pending_approval`.

Prepended to most agent system prompts via `withAssistantPolicy()` .

11.2 Autonomous Agent Planner System Prompt

Hermes autonomous operator on the user's Mac. No Gmail/API keys – use `browser.goto` + `screen.observe`.
 Services: Gmail `https://mail.google.com`, X `https://x.com`, LinkedIn `https://www.linkedin.com/feed/`, Calendar, GitHub.
 JSON only:

```
{
  "think": "...",
  "action": "continue|finish|blocked|ask_user",
  "tool": "...",
  "payload": {},
  "summary": "...",
  "final": "...",
  "question": "..."
}
```

- `continue`: ONE tool. After `browser.goto` the system auto-captures screen.
- `finish`: include final answer for user.
- `blocked|ask_user`: set question when login/CAPTCHA/human-only step; never guess passwords.
- Use `code.self_edit` or `tools.author` when capability is missing.
- High-risk sends/posts still require approval (do not bypass).

`max {max_steps} steps; prefer action over refusal.`

11.3 Planner User Prompt Template

```
Goal: {goal}
{steering hints}
Tools: {tool catalog}
Last error: {lastError}
Latest screen: {capturePath}
Prior steps:
{trace}
Next single tool, or finish, or blocked with question?
```

11.4 Intent Classifier System Prompt

You are an intent classifier for a personal assistant. Pick exactly one intent id from the catalog.
 Prefer research for shopping/links/best-product questions; prefer `laptop_control` for opening sites in Arc.
 Respond with ONLY valid JSON, no markdown prose:

```
{
  "intent": "<id>",
  "confidence": 0.0-1.0,
  "reasoning": "brief",
  "entities": {...}
}
```


 Optional entities: `approvalAction`, `approvalId`, `editText`, `assistantControl`, `researchContinue`, `researchEnd`, `toolName`, `payloadText`, `url`, `memoryAction`, `memoryId`

Routing rules:

- Any Gmail, email, calendar, Slack, Notion, Amazon, X/Twitter → laptop_control
- Do not assume an API exists; laptop_control uses screen + browser only

Catalog: [from configs/intents.yaml]

Assistant state: {running|paused|emergency_stop}

Pending approval ids: ...

Active research topic: ...

Classifier model: @cf/meta/llama-3.2-3b-instruct

11.5 Screen Vision Prompts

Describe (Llama 3.2 Vision):

This is a screenshot of the user's {service} in a web browser (Arc on macOS).

Describe everything useful for a personal assistant:

- unread or important messages, sender names if visible
- deadlines, event names, errors, login walls
- anything waiting for the user to reply or act

Be specific and factual. Plain text only, no JSON.

Structure (text model):

Given this screen description from the user's Mac, extract operational facts.

Reply ONLY valid JSON (no markdown):

```
{"summary":"2-4 sentences","openLoops":["..."],"risks":["..."]}
```

If nothing actionable, use empty arrays. Do not invent names not in the description.

Vision model: @cf/meta/llama-3.2-11b-vision-instruct via /ai/run/

11.6 Research Agent

Initial:

Research analyst: cite sources, state assumptions, give confidence (low/medium/high), and a concrete recommended next action.

Follow-up:

You are continuing a research thread. Honor follow-ups (links, personalization from memory). Include https:// links when relevant.

11.7 General Agent

You are Hermes Personal OS – a local-first assistant with approval-gated tools. Use stored preferences; do not re-ask for facts in memory.

11.8 Coding Agent

You are a senior engineer working on the Hermes Personal OS monorepo. Propose concrete

file-level changes; use `code.self_edit` when execution is available.

11.9 Tools Author

Design a reusable tool macro for Hermes Personal OS using ONLY existing built-in tools.
Reply ONLY JSON: {"name":"custom.slug","description":"...","steps":[...]}
Macro name MUST start with "custom.".

11.10 Steering injection (runtime)

User course-correction (keep browser/tabs as-is, replan next step): {user message}

12. Configuration Files

File	Purpose
configs/agent-loop.yaml	Autonomous loop limits
configs/cloudflare-models.yaml	Model routing per task type
configs/risk-policy.yaml	Tool risk + approval rules
configs/intents.yaml	Intent catalog for classifier
configs/autonomy-policy.yaml	Proactive scan, self-edit when stuck
configs/proactivity-policy.yaml	Notification scoring
configs/memory-policy.yaml	Memory store rules

Cloudflare model routes

Task	Model
default	@cf/zai-org/glm-4.7-flash
research	@cf/openai/gpt-oss-20b
coding	@cf/qwen/qwen2.5-coder-32b-instruct
vision	@cf/meta/llama-3.2-11b-vision-instruct
memory/status	@cf/meta/llama-3.2-3b-instruct

13. Proactive & Background

File: apps/orchestrator/src/proactive-scheduler.ts

Started from: iMessage bridge (not chat-server)

Interval: PROACTIVE_SCAN_INTERVAL_MS (default 15 min)

Each tick:

1. Skip if assistant not `running` or hindrance active
2. `runPresenceScan()` — one digital surface (Gmail/X/LinkedIn/Calendar round-robin)
3. Morning brief at hour 7
4. Evening review at hour 21
5. `ProactiveScanner` — score $\geq 70 \rightarrow$ iMessage notification

Digital presence flow:

```
browser.goto(service URL) → wait 3s → screen.observe → vision → context graph
```

14. iMessage Bridge

Not in default `pnpm workspace scripts` — code remains in `apps/imessage-bridge/` .

Safeguards:

- **Allowlist only:** `APPROVED_IMESSAGE_SENDERS` + `IMESSAGE_DEFAULT_RECIPIENT`
- **Automated SMS filter:** OTP, Apple ID codes, `(smsft)` senders ignored
- **No reply** to bank/Apple short codes

15. Environment Variables

Variable	Purpose
HERMES_OS_ROOT	Workspace root
DATABASE_PATH	SQLite path
CLOUDFLARE_ACCOUNT_ID / CLOUDFLARE_API_TOKEN	Workers AI (required for agents)
HERMES_CHAT_PORT	Chat UI port (3847)
HERMES_ACTIVITY_CONSOLE	Log activity to stdout
HERMES_ACTIVITY_JSONL	Set 0 to disable JSONL
HERMES_API_URL / HERMES_API_KEY	Hermes Python gateway
HERMES_DEFAULT_BROWSER	Arc (default)
SUPERMEMORY_API_KEY	Semantic memory
APPROVED_IMESSAGE_SENDERS	iMessage allowlist JSON
IMESSAGE_DEFAULT_RECIPIENT	Your phone/email
HERMES_DISABLE_PRESENCE_SCAN	Disable proactive scans
HERMES_CHAT_NO_OPEN	Don't auto-open browser on chat start

16. Data Files

Path	Contents
data/local.sqlite	Context graph, audit, approvals, memories
data/activity.jsonl	Live activity log
data/pending-blocked.json	Autonomous agent paused for user
data/pending-hindrance.json	System hindrance (vision, chat.db)
data/pending-login.json	Credential reply session
data/custom-tools/	User-defined tool macros

17. File Index

Component	Path
Bootstrap	apps/orchestrator/src/system.ts
Orchestrator	apps/orchestrator/src/orchestrator.ts
AutonomousAgent	apps/orchestrator/src/autonomous-agent.ts
Run manager	apps/chat-server/src/run-manager.ts
Chat server	apps/chat-server/src/server.ts
Chat UI	apps/chat-server/public/
Tail logs	scripts/tail-logs.mjs
Intent classifier	packages/shared/src/intent/intent-classifier.ts
Assistant policy	packages/shared/src/assistant-policy.ts
Steering classifier	apps/orchestrator/src/steering-classifier.ts
SteerController	packages/shared/src/steer-controller.ts
Screen context	packages/tool-executor/src/screen-context.ts
Tool registry	packages/tool-executor/src/create-tool-registry.ts
Activity monitor	packages/audit-log/src/activity-monitor.ts
Proactive scheduler	apps/orchestrator/src/proactive-scheduler.ts

End of Hermes Personal OS Implementation Guide